

ALGORITMA DAN STRUKTUR DATA(ASD)
TUGAS PRAKTIKUM 7 ALGORITMA SEARCHING



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – ALGORITMA SEARCHING

Implementasikan program algoritma Searching Sequential, Sequential Sorted, dan Binary. Tuliskan source code, hasil running program, dan analisis. Buat kesimpulan hasil praktikum.

Jawaban:

Kode Sequential Search:

```
1. #include <stdio.h>
2.
3. void pencarian_sekuensial(int data[], int n, int key);
4.
5. int main() {
6.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
7.     int n = 10;
8.
9.     pencarian_sekuensial(data, n, 48);
10.
11.     return 0;
12.}
13.
14. void pencarian_sekuensial(int data[], int n, int key) {
15.     int i;
16.     for (i = 0; i < n; i++) {
17.         if (data[i] == key) {
18.             printf("data ditemukan pada indeks %d\n", i);
19.             return;
20.         }
21.     }
22.     printf("data tidak ditemukan\n");
23.}
```

Hasil Kode:

```
data ditemukan pada indeks 3
```

Analisis Kode:

Pada kode pencarian sekuensial, program bekerja dengan cara yang paling sederhana, yaitu memeriksa data satu per satu dari urutan paling awal sampai akhir. Program menggunakan satu perulangan untuk mencocokkan setiap nilai dengan angka target yang sedang dicari. Kelebihan utama dari kode ini adalah kemudahannya, di mana kita tidak perlu repot mengurutkan data terlebih dahulu. Namun sisi buruknya, jika jumlah datanya sangat banyak dan angka yang kita cari berada di urutan paling belakang atau bahkan tidak ada sama sekali, program akan memakan tenaga dan waktu yang lama karena harus memeriksa seluruh deretan angka tanpa terkecuali.

Kode Sorted Sequential Search:

```
1. #include <stdio.h>
2.
3. void urutkan_data(int data[], int n);
4. void sekuensial_berurutan(int data[], int n, int key);
5.
6. int main() {
7.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
8.     int n = 10;
9.
10.    urutkan_data(data, n);
11.    sekuensial_berurutan(data, n, 50);
12.
13.    return 0;
14.}
15.
16.void urutkan_data(int data[], int n) {
17.    int i, j, sementara;
18.    for (i = 0; i < n - 1; i++) {
19.        for (j = 0; j < n - i - 1; j++) {
20.            if (data[j] > data[j + 1]) {
21.                sementara = data[j];
22.                data[j] = data[j + 1];
23.                data[j + 1] = sementara;
24.            }
25.        }
26.    }
27.}
28.
29.void sekuensial_berurutan(int data[], int n, int key) {
30.    int i;
31.    for (i = 0; i < n; i++) {
32.        if (data[i] > key) {
33.            printf("pencarian berhenti di indeks %d\n", i);
34.            printf("data tidak ditemukan\n");
35.            return;
36.        }
37.        if (data[i] == key) {
38.            printf("data ditemukan pada indeks %d\n", i);
39.            return;
40.        }
41.    }
42.    printf("data tidak ditemukan\n");
```

43.}

Hasil Kode:

pencarian berhenti di indeks 2

Analisis Kode:

Untuk kode pencarian sekuensial berurutan, cara kerjanya mirip dengan pencarian biasa namun memiliki trik yang lebih pintar karena memanfaatkan data yang sudah diurutkan dari kecil ke besar. Di dalam perulangan kode ini, terdapat syarat tambahan di mana program akan langsung berhenti bekerja apabila nilai yang sedang dibaca sudah lebih besar dari angka target. Langkah ini sangat efisien dan menghemat waktu karena secara logika tidak mungkin angka target ditemukan di sisa barisan selanjutnya. Kode ini sangat membantu jika kita ingin menghindari pemeriksaan data sampai akhir ketika targetnya memang tidak ada di dalam daftar tersebut.

Kode Binary Search:

```
1. #include <stdio.h>
2.
3. void urutkan_data(int data[], int n);
4. void pencarian_biner(int data[], int n, int key);
5.
6. int main() {
7.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
8.     int n = 10;
9.
10.    urutkan_data(data, n);
11.    pencarian_biner(data, n, 76);
12.
13.    return 0;
14.}
15.
16. void urutkan_data(int data[], int n) {
17.     int i, j, sementara;
18.     for (i = 0; i < n - 1; i++) {
19.         for (j = 0; j < n - i - 1; j++) {
20.             if (data[j] > data[j + 1]) {
21.                 sementara = data[j];
22.                 data[j] = data[j + 1];
23.                 data[j + 1] = sementara;
24.             }
25.         }
26.     }
27.}
28.
29. void pencarian_biner(int data[], int n, int key) {
```

```

30.     int bawah = 0, atas = n - 1, tengah;
31.
32.     while (bawah <= atas) {
33.         tengah = bawah + (atas - bawah) / 2;
34.
35.         if (data[tengah] == key) {
36.             printf("data ditemukan pada indeks %d\n", tengah);
37.             return;
38.         } else if (data[tengah] < key) {
39.             bawah = tengah + 1;
40.         } else {
41.             atas = tengah - 1;
42.         }
43.     }
44.     printf("data tidak ditemukan\n");
45. }

```

Hasil Kode:

```
data ditemukan pada indeks 5
```

Analisis Kode:

Pada kode pencarian biner, program bekerja dengan strategi memotong rentang pencarian menjadi dua bagian secara terus-menerus. Aturan mutlak dari kode ini adalah data wajib diurutkan terlebih dahulu sebelum pencarian dimulai. Program akan selalu mengambil nilai tepat di tengah barisan data dan membandingkannya dengan angka target. Jika targetnya ternyata lebih kecil, maka separuh data di bagian kanan akan langsung diabaikan, begitu pula sebaliknya. Cara kerja ini membuat proses pencarian menjadi sangat cepat dan ringan, terutama untuk tumpukan data yang sangat banyak, karena program bisa langsung melompati angka-angka yang sudah pasti tidak relevan.

Kesimpulan:

Berdasarkan analisis dari ketiga metode pencarian di atas, dapat disimpulkan bahwa tidak ada satu pun algoritma yang paling sempurna untuk segala situasi, karena pemilihannya sangat bergantung pada kondisi dan jumlah data yang kita miliki. Jika kita berhadapan dengan data yang masih acak atau berantakan, pencarian sekuensial biasa adalah pilihan yang paling masuk akal dan praktis untuk digunakan. Meskipun cara kerjanya sangat sederhana dan berpotensi memakan waktu lebih lama jika datanya banyak, metode ini membebaskan sistem dari beban kerja tambahan karena kita tidak perlu bersusah payah mengurutkan deretan angka tersebut terlebih dahulu sebelum mulai mencari.

Di sisi lain, jika kita sudah memiliki sekumpulan data yang terurut rapi dari kecil ke besar, sangat disarankan untuk beralih menggunakan pencarian biner atau pencarian sekuensial berurutan. Pencarian biner menjadi juara utama dalam hal kecepatan dan efisiensi, terutama untuk tumpukan data yang sangat besar, karena kemampuannya

membelah area pencarian membuat program tidak perlu mengecek setiap angka satu per satu. Sementara itu, pencarian sekuensial berurutan bisa menjadi jalan tengah yang cerdas, karena fitur pemberhentian awalnya sangat membantu sistem untuk langsung menyudahi pencarian ketika secara logika angka yang kita cari sudah pasti tidak mungkin ada di sisa daftar tersebut.